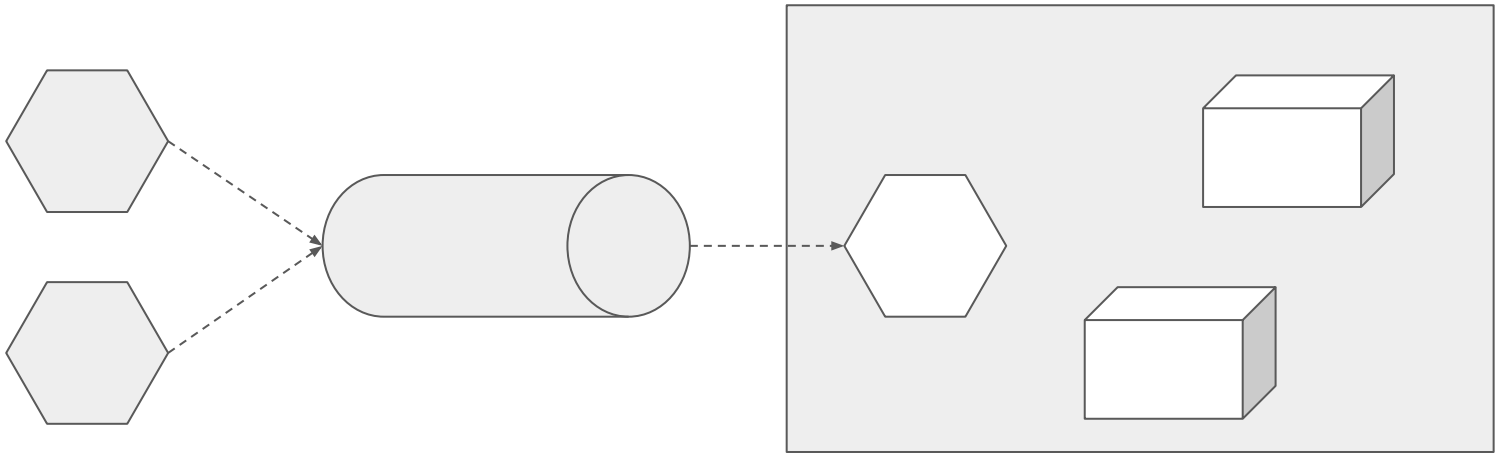
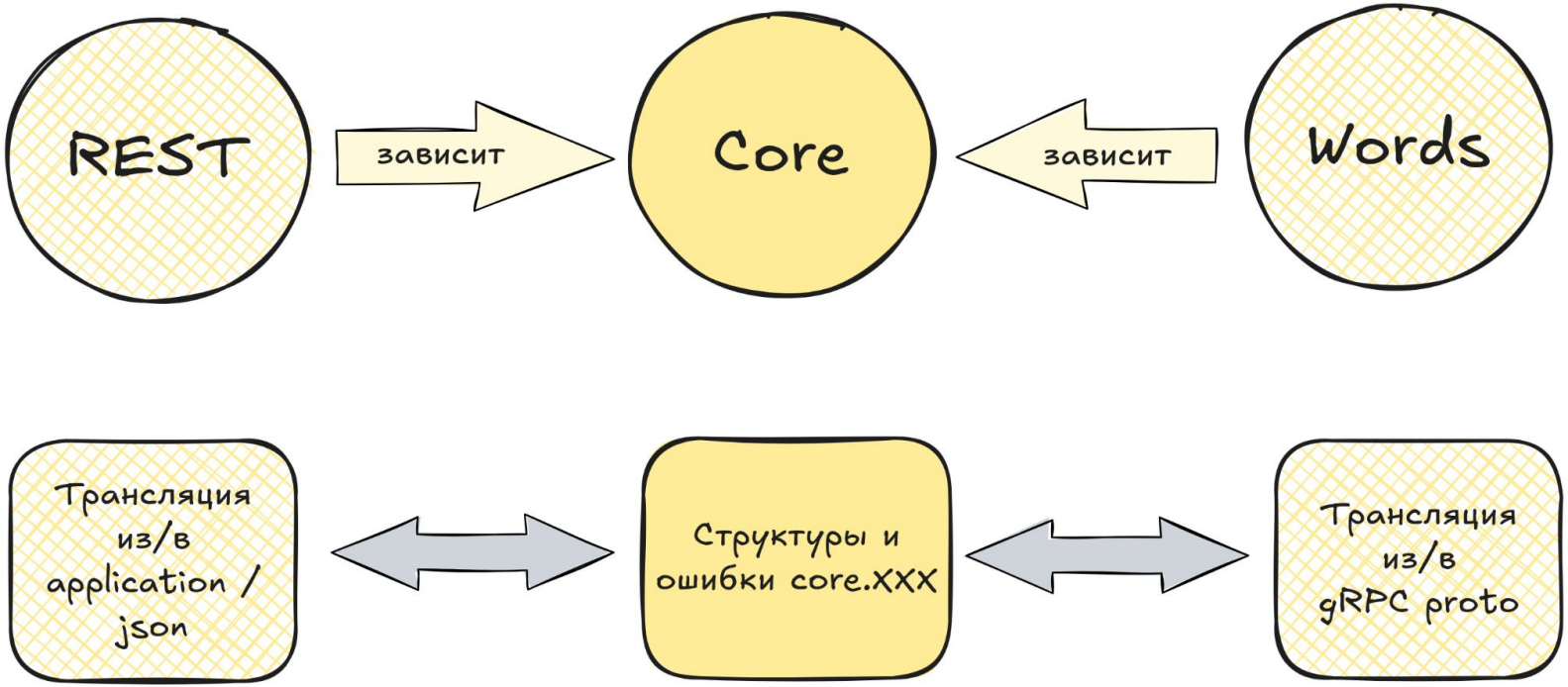


Архитектура ПО



Часто встречающиеся ошибки и замечания

адаптеры не должны зависеть друг от друга, только от core/*



Что такое Архитектура ПО?

Архитектура ПО — это высокоуровневая структура системы:

- компоненты и их взаимодействие- пользовательские интерфейсы, сервисы, базы данных, протоколы связи
- 4 измерения …

Что такое Архитектура ПО?

Измерения в архитектуре ПО

- Архитектурные характеристики (масштабируемость, производительность,...)

Что такое Архитектура ПО?

Измерения в архитектуре ПО

- Архитектурные характеристики (масштабируемость, производительность,...)
- Архитектурные решения (ADR)

Что такое Архитектура ПО?

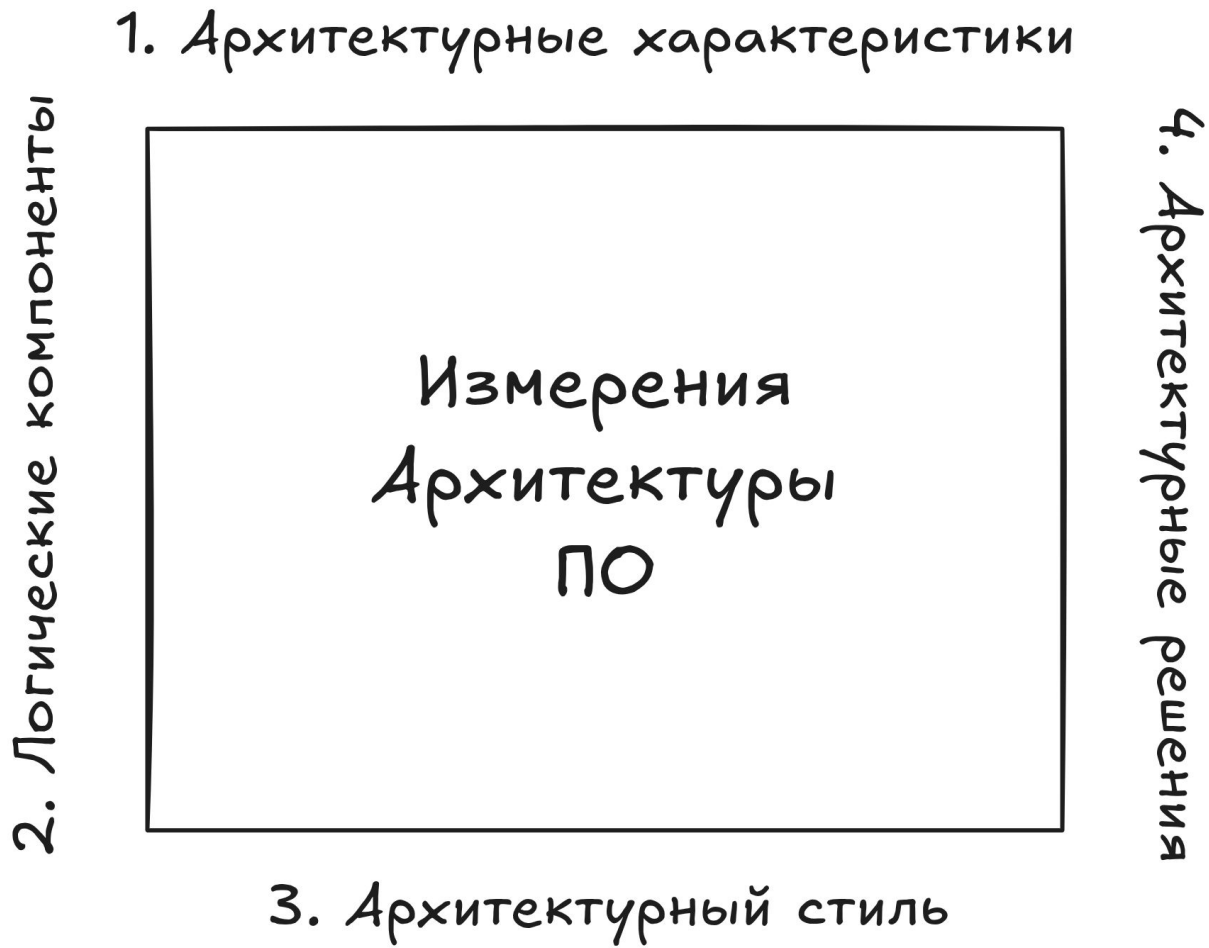
Измерения в архитектуре ПО

- Архитектурные характеристики (масштабируемость, производительность,...)
- Архитектурные решения (ADR)
- Логические компоненты (функциональные блоки и их взаимодействие)

Что такое Архитектура ПО?

Измерения в архитектуре ПО

- Архитектурные характеристики (масштабируемость, производительность,...)
- Архитектурные решения (ADR)
- Логические компоненты (функциональные блоки и их взаимодействие)
- Архитектурный стиль (общая форма и структура системы)

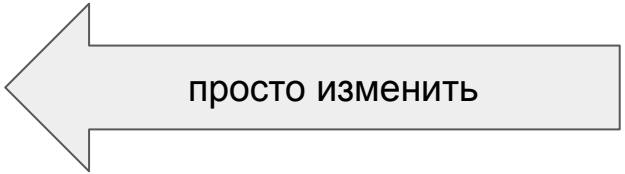

1. Архитектурные характеристики
2. Логические компоненты
Измерения
Архитектуры
ПО
4. Архитектурные решения
3. Архитектурный стиль

Архитектура ПО vs Дизайн ПО

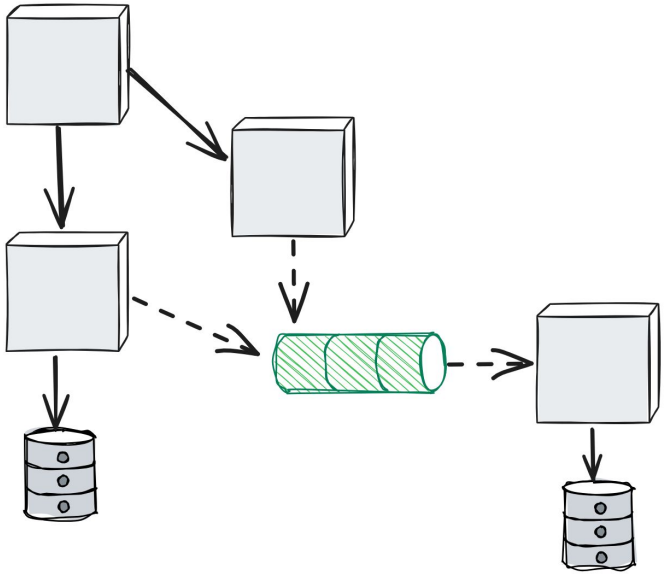
- Архитектура
  - стены
  - фундамент
  - количество этажей
  - тип крыши

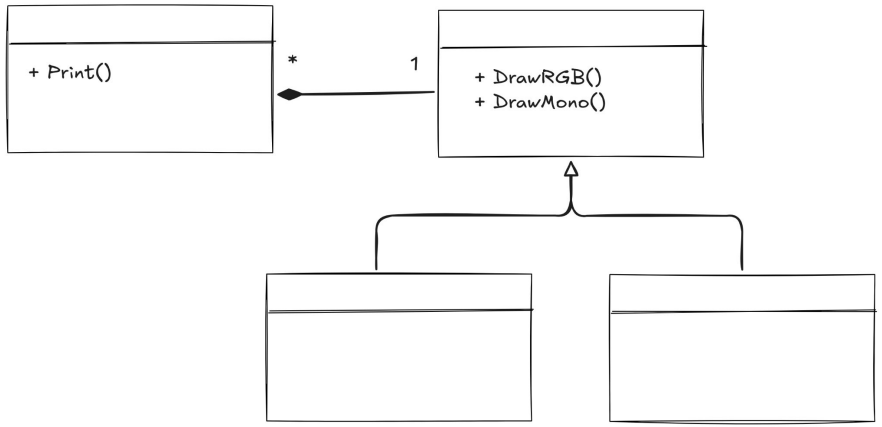


- Дизайн
  - цвет крыши
  - облицовка стен
  - куда повесить телевизор?
  - подсветка в ванной



Архитектура ПО vs Дизайн ПО





Архитектура vs Дизайн

архитектура

дизайн



	Архитектура	Дизайн
Участники	много	1-2
Решение	стратегическое	тактическое
Время решения	дни-недели	минуты-часы
Решение или изменения	очень сложно	легко
Компромиссы	более значимые	менее значимые

Архитектура или дизайн?

1. Физическая структура кода?
2. Стратегическое или тактическое?
3. Сколько времени на размышление (час или неделя)?
4. Сколько человек принимают решение (1 разраб или несколько отделов)?
5. Долгосрочная перспектива или краткосрочное решение?
6. Большие или малые усилия (спринт или год)?
7. Значимые или незначимые компромиссы (масштабируемость или простота, кнопка синяя или зеленая)?

System Design

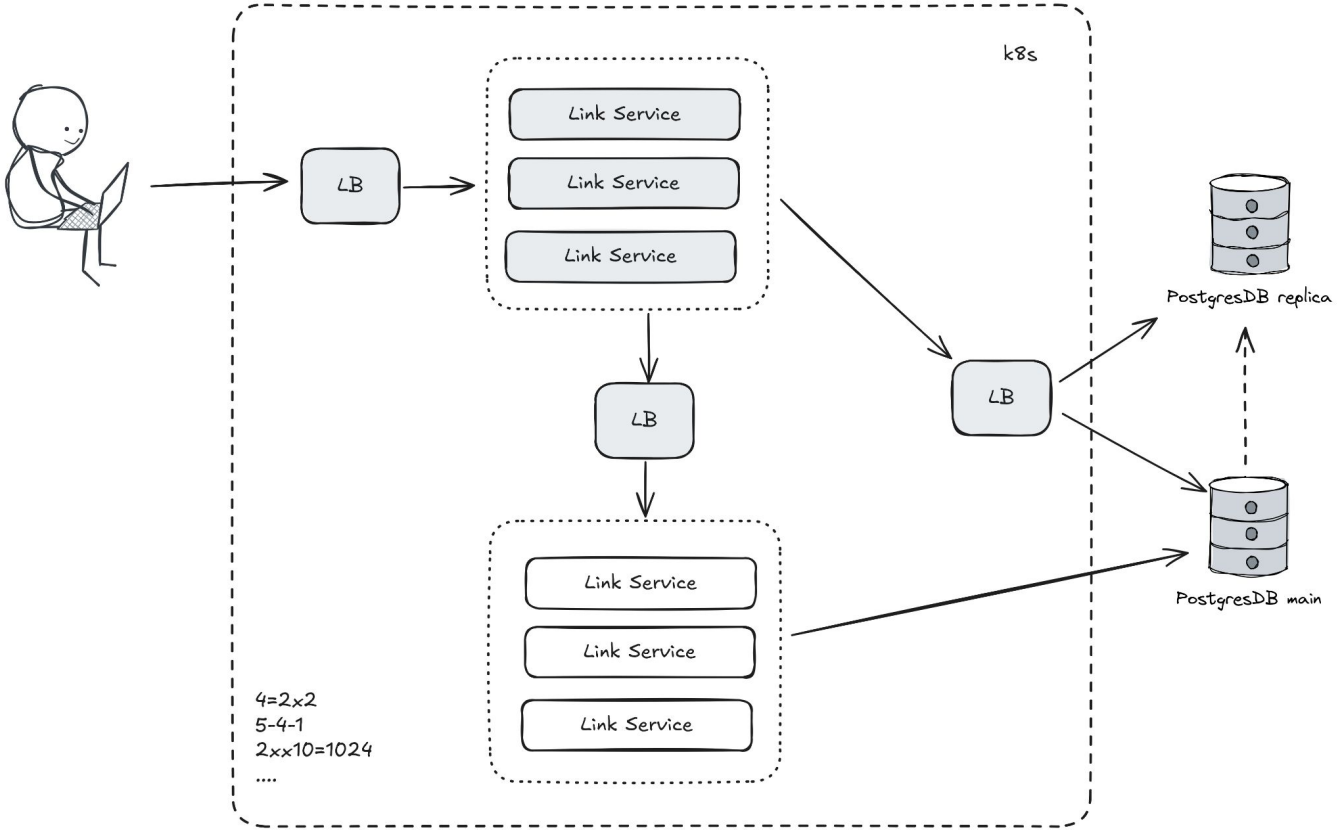
Детализация архитектуры, тщательная проработка отдельных модулей и их внутренней структуры.

Цель: создать рабочий план реализации

- детальное проектирование компонентов и их интерфейсов;
- разработку схем баз данных (таблицы, связи, ограничения);
- проектирование пользовательских интерфейсов;
- выбор конкретных технологий и инструментов;
- проработку алгоритмов и структур данных;
- описание протоколов взаимодействия между компонентами.

System Design (url shortener)





Законы Архитектуры ПО

Первый закон архитектуры ПО – **все в архитектуре основано на компромиссах**

- у каждого решения есть плюсы и минусы
- нет лучших решений, есть оптимальные для наших требований и приоритетов

производительность
простота

надежность
безопасность

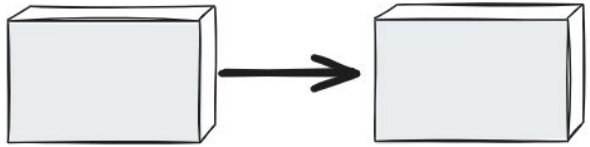


Законы Архитектуры ПО

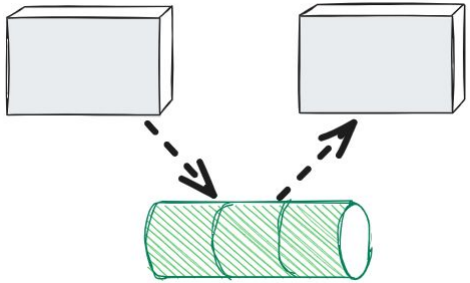
Второй закон архитектуры ПО - **«почему» намного важнее «как»**

- почему был выбор того или иного решения
- какие компромиссы рассматривались
- знания будущим мейнтейнерам и себе
- **«как»** - удел дизайна (можно посмотреть на решение/код, если что)

Пример - синхронная связь или асинхронная?



Плюсы	Минусы
согласованность	расширяемость
обработка ошибок	отказоустойчивость
транзакции	отзывчивость



Плюсы	Минусы
отзывчивость	согласованность
расширяемость	обработка ошибок
отказоустойчивость	транзакции

Архитектурные характеристики

- нефункциональные требования, возможности, критически важные или значимые для успешной работы системы:

  Производительность, Доступность, Масштабируемость, Расширяемость, Гибкость, Интегрируемость, Отказоустойчивость, Осуществимость, Простота, Модульность, Тестируемость, Развертываемость, Связанность, Защищенность, Переносимость, Локализация, Восстанавливаемость, Устойчивость, Законность, Юзабилити, Аутентификация, Авторизация, Наблюдаемость, Целостность данных, Согласованность данных, Адаптируемость, Конфигурируемость, Настраиваемость, Контролируемость …

  и еще тысячи других.

Архитектурные характеристики

Откуда получать архитектурные характеристики?

- Из предметной области (требования)
- Знание окружения (приоритеты организации)
- Целостное знание о предметной области (опыт, правила)

Архитектурные характеристики

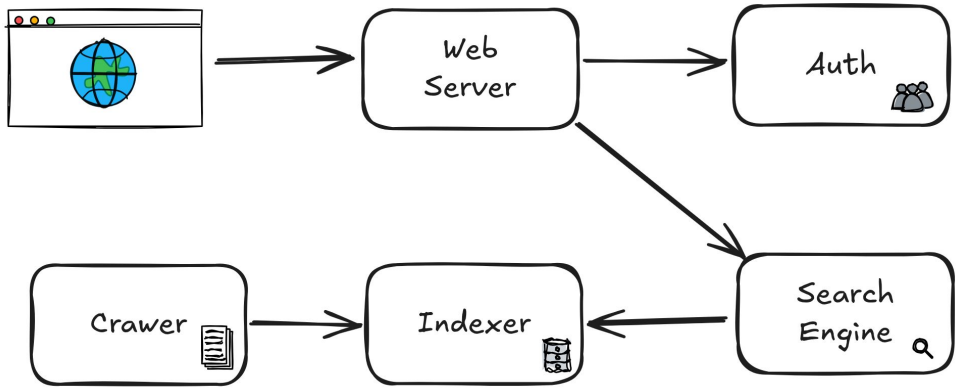
Откуда получать архитектурные характеристики?

- Из предметной области (требования)
- Знание окружения (приоритеты организации)
- Целостное знание о предметной области (опыт, правила)

Много характеристик – слишком сложная архитектура, выбираем не больше семи (5-7). Характеристики часто взаимосвязаны.

Логические компоненты

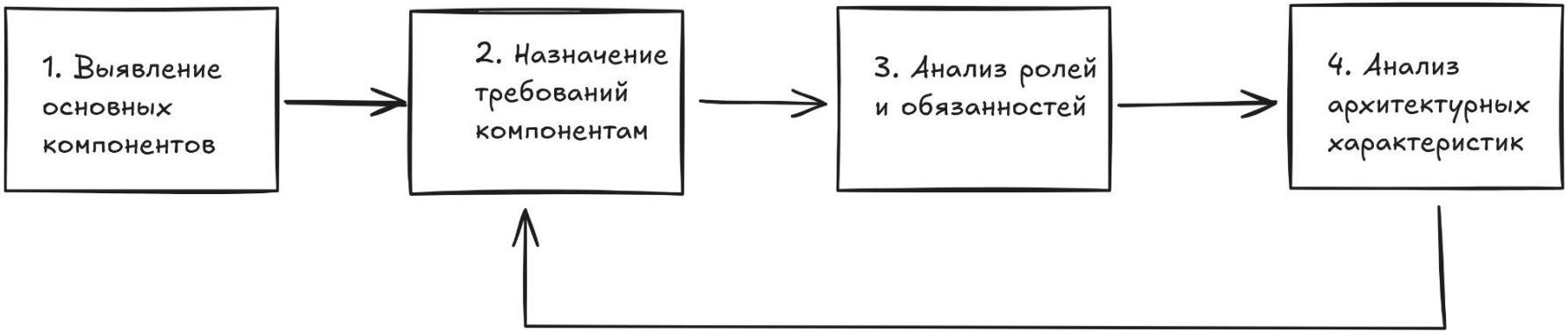
- функциональные строительные блоки, образующие предметную область
- содержат исходные код для каждой бизнес-функции (структура каталогов исходного кода часто отражает компоненты, а сам код - уже часть дизайна ПО).
- логическая архитектура определяет компоненты и их взаимодействие (связанность)



Как найти логические компоненты?

1. начальный набор основных компонентов
    - на основе рабочих процессов (шаги → компоненты)
    - на основе субъектов и действий (участники → действия → компоненты)
2. распределение обязанностей
    - компонент отвечает за определенную функцию
3. убеждаемся, что компонент справляется со своими обязанностями
    - должен отвечать за то, что ему назначили?
    - не слишком много обязанностей?
    - связность? (не связанность!)
4. подходит под архитектурные характеристики?

Как найти логические компоненты?



Как найти логические компоненты?

Ловушка для сущностей:

- компонент с общим названием типа "менеджер", "управление", "координация", "супервизор", "контроллер", "агент", "процессор"
- обычно принимает на себя слишком много обязанностей
- название ловушки не говорит, что конкретно делает компонент (папка utils 😉).

Связанность

- степень зависимости компонентов друг от друга

Типы связей

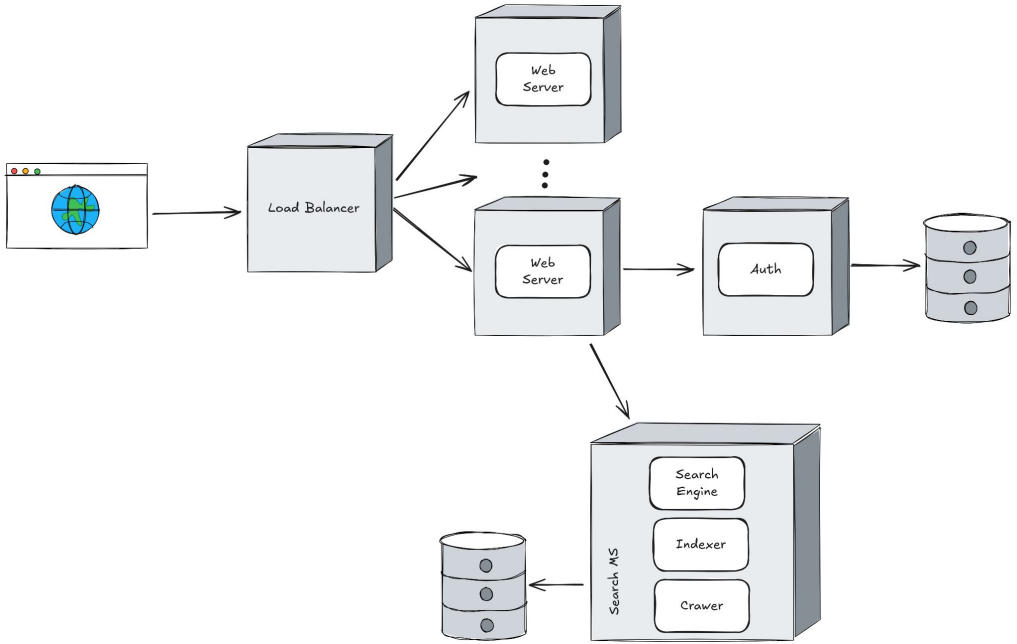
- афферентные (входящие)
- эфферентные (исходящие)

Уровень связанности CT = CA + CE

Меньше связанность - меньше зависимостей, но сложнее управление рабочими процессами.

Физическая архитектура

- определяет архитектурный стиль, сервисы, базы данных, протоколы

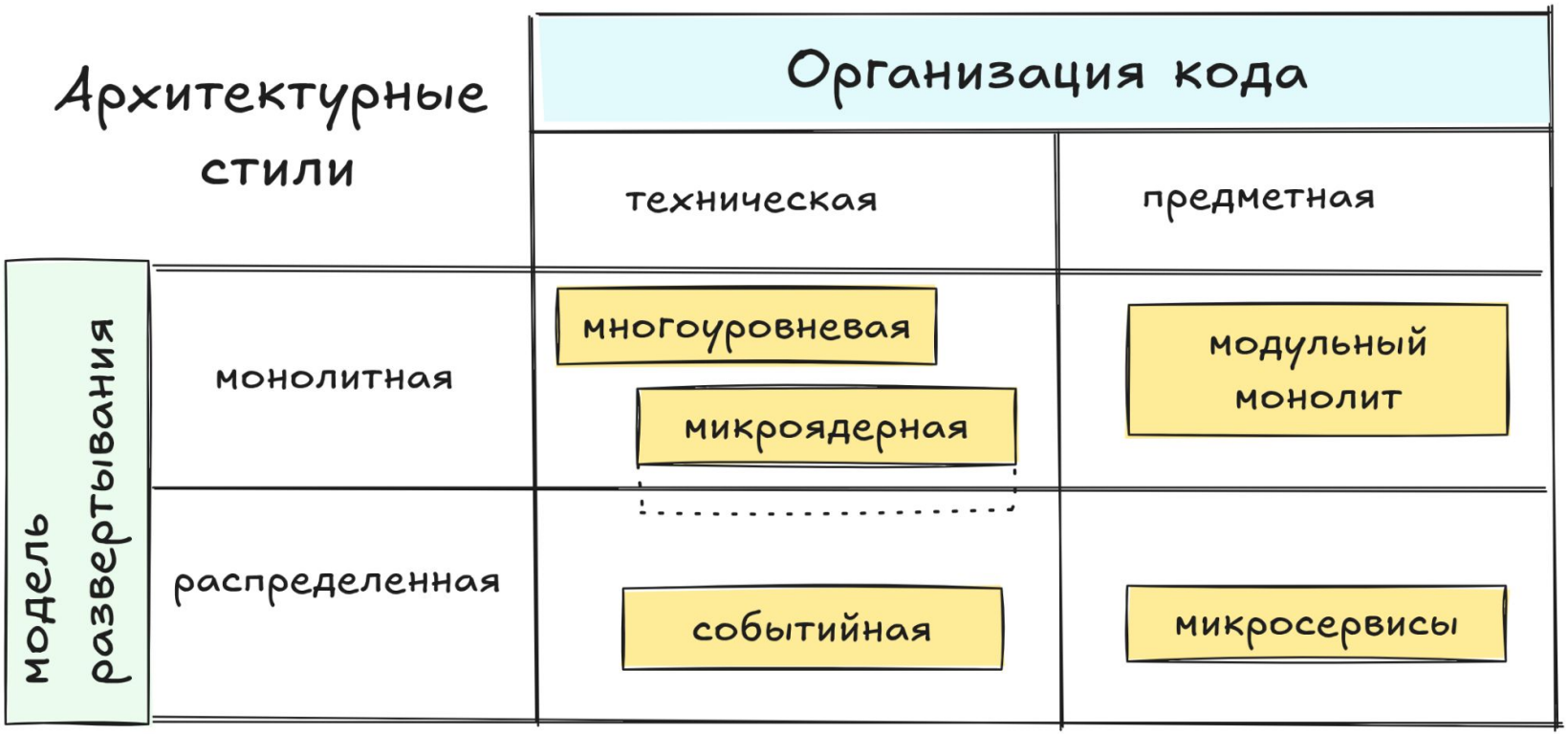


Архитектурные стили

Архитектурный стиль

- форма, структура, уникальный набор характеристик.
- выбирается с помощью архитектурных характеристик и логических компонентов после анализа компромиссов.

Классификация архитектурных стилей



Архитектура с технической организацией кода

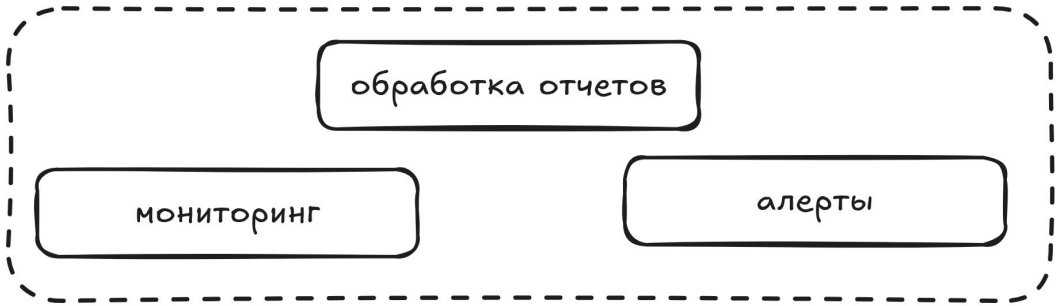
Разделение по техническим задачам:




```
app/
    server/
        reports/
        alerts/
    service/
        reports/
        alerts/
    storage/
        reports/
        alerts/
```

Архитектура с предметной организацией кода

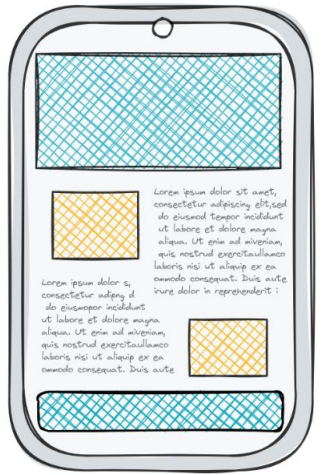
Разделение по предметным областям и задачам:



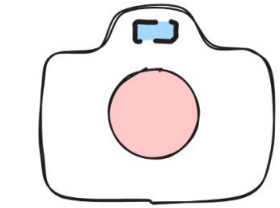

```
app/
    reports/
        server/
        service/
        storage/
    alerts/
        server/
        service/
        storage/
```

Что лучше, монолиты или микросервисы?

Что лучше, монолиты или микросервисы?

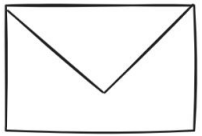


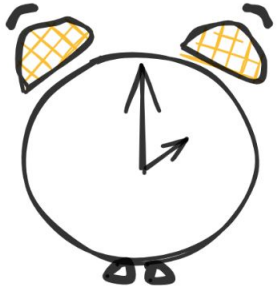
Smartphone













Компромиссы монолитов

Плюсы	Минусы
• простота разработки в начале • дешевле в начале проекта • осуществимость – быстрый старт • легкая отладка • надежность – мало "подвижных" частей, (почти) не использует сеть • развертываемость - один "бинарь"/докер	• масштабирование – все или ничего • надежность – одна ошибка и все! • развиваемость – чем дальше катимся, тем больше ком; невозможно использовать разнообразные языки • развертываемость – любое изменение есть полная замена

Компромиссы распределенной модели

Плюсы	Минусы
● масштабируемость ● производительность ● модульность ● тестируемость ● развертываемость ● отказоустойчивость	● производительность – задержки сети, большое время отклика ● стоимость – инфраструктура, поддержка ● сложность разработки ● сложность отладки

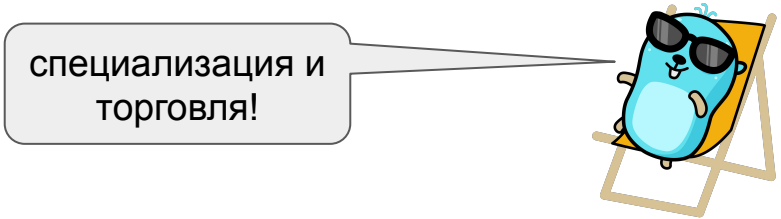
Заблуждения о распределённых вычислениях

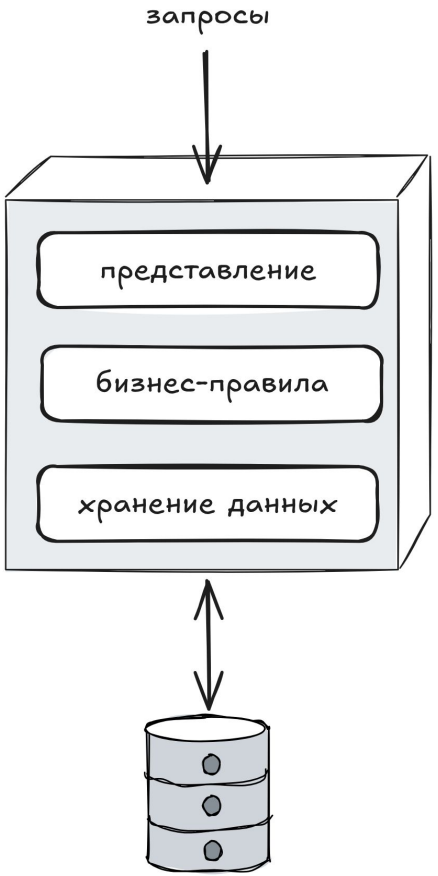
1. Сеть надёжна
2. Задержка равна нулю
3. Пропускная способность бесконечна
4. Сеть защищена
5. Топология не меняется
6. Есть один администратор
7. Транспортные расходы равны нулю
8. Сеть однородна

Wikipedia

Многоуровневая Архитектура

- Техническое разделение
  - представление
  - бизнес-логика
  - хранение... кеш
  - аутентификация
- Обычно разворачивается как монолит (может быть и распределенной)
- Уровней может быть сколько угодно.





Компромиссы многоуровневой архитектуры

Сильные стороны	Слабые стороны
● Осуществимость (время, бюджет) ● Техническое разделение (специализация команд) ● Обработка данных (одна оптимизированная бд) ● Производительность (нет сети между частями) ● Скорость разработки ● Компактность и эффективность	● Масштабируемость (монолит) ● Эластичность (монолит) ● Развертываемость (замена куском) ● Тестируемость (высокая связанность) ● Месиво кода при росте

Компромиссы многоуровневой архитектуры

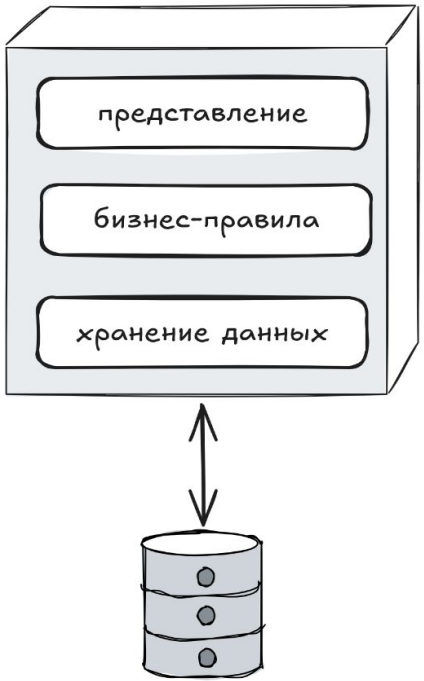
Сильные стороны	Слабые стороны
• Осуществимость (время, бюджет) • Техническое разделение (специализация команд) • Обработка данных (одна оптимизированная бд) • Производительность (нет сети между частями) • Скорость разработки • Компактность и эффективность	• Масштабируемость (монолит) • Эластичность (монолит) • Развертываемость (замена куском) • Тестируемость (высокая связанность) • Месиво кода при росте

Многоуровневые архитектуры упрощают изменения в технической части (другая ДБ или UI), но сильно затрудняют в предметной.
Если постоянно развивается или изменяется предметная область, то лучше использовать другую архитектуру.

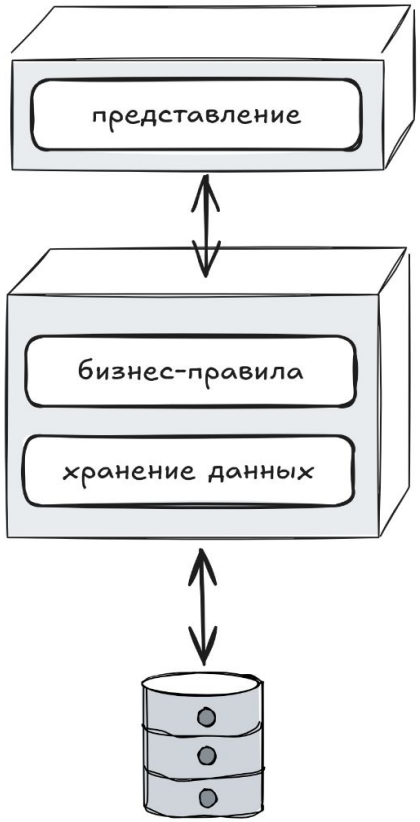
Многоуровневая Архитектура - "физика"

Логический многоуровневый монолит может быть реализован в виде различных физических архитектур

двухуровневая архитектура
(настольные приложения)



трехуровневая архитектура
(веб сервисы)



втраиваемая архитектура
(мобильная)

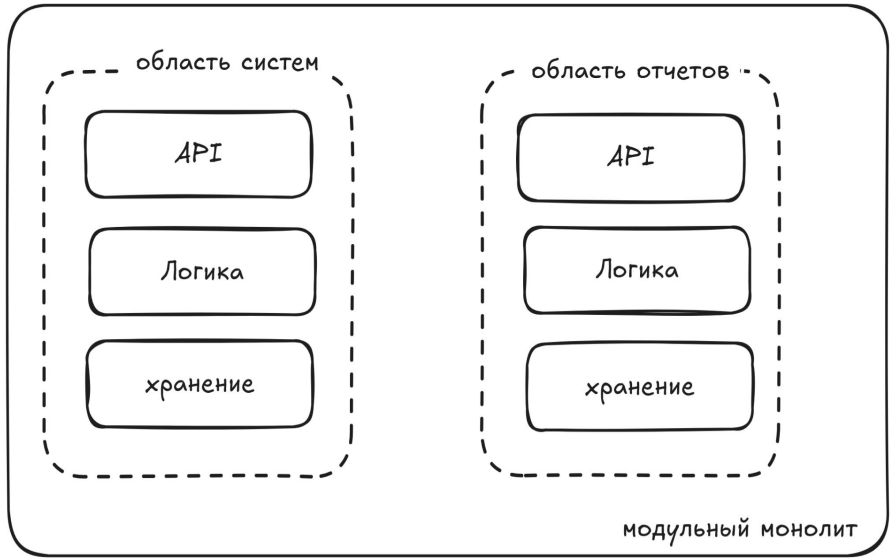


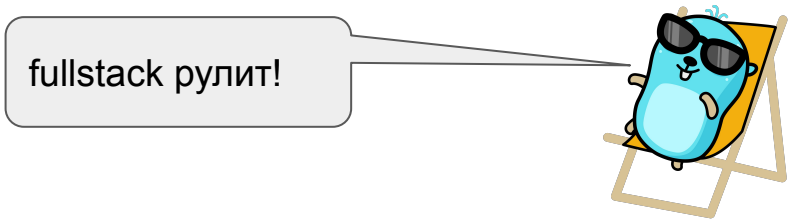
Компромиссы физических архитектур

Архитектура	Плюсы	Минусы
Двухуровневая	• Богатый UI • Производительность • Простота	• Средняя масштабируемость • Увеличение сложности по мере развития • Средняя надежность
Трехуровневая	• Отдельный UI • Высокая масштабируемость • Плюсы распределенной архитектуры	• Меньшая надежность • Сложность • Минусы распределенной архитектуры
Встраиваемая	• Автономность • Единый технологический стек • Адаптация к устройствам	• Меньшая масштабируемость • Ограниченность ресурсов • Привязка к платформе

Модульный монолит

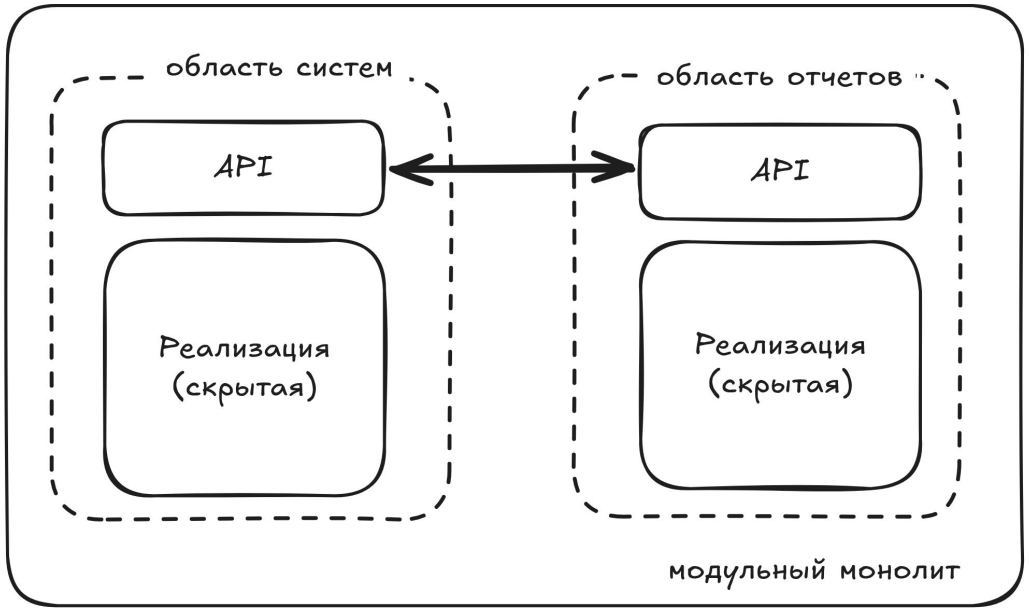
- Каждый модуль монолита представляет часть предметной области и содержит все необходимые для нее бизнес-функции.
- Приложение организуется не в виде горизонтальных уровней для решения технических задач, а в виде вертикальных сегментов.
- Модульные монолиты разрабатываются кросс-функциональными командами, специализация - предметные области.





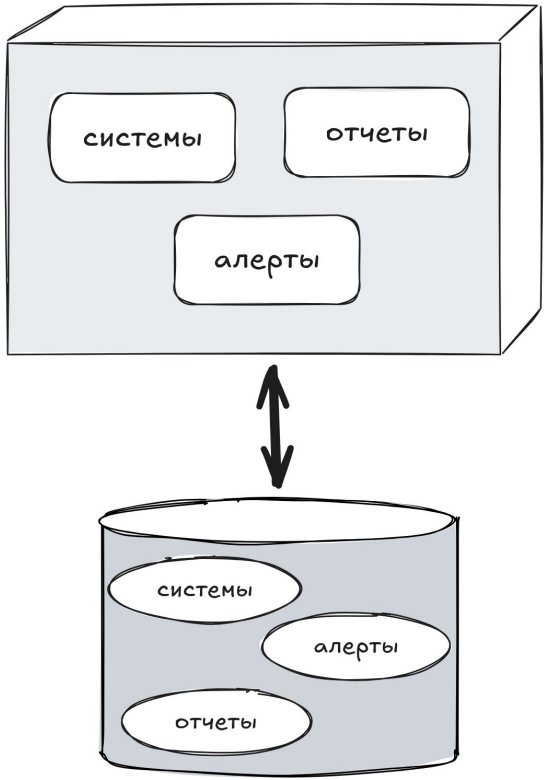
Модульный монолит - коммуникация

Коммуникация между модулями - **исключительно через API**:



Модульный монолит - база данных

- База - монолитная
- Схемы / наборы таблиц у каждого модуля свои
- Чтение чужих данных только через API, **никаких JOIN-ов таблиц разных модулей**

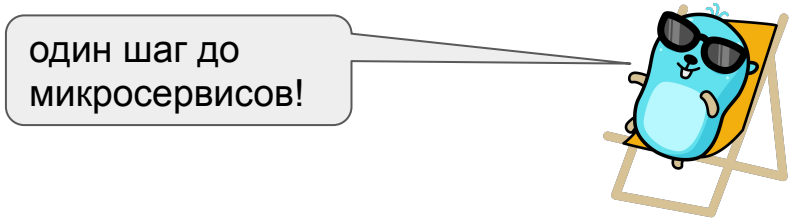


Компромиссы модульного монолита

Сильные стороны	Слабые стороны
• Разделение предметных областей • Предметная специализация (спецы по алертам, отчетам, ..) • Производительность (нет задержек сети) • Удобство сопровождения (проще код, меньше шансов "наломать дров") • Тестируемость	• Сложно переиспользовать общую логику (вынос в библиотеку и как следствие большая связанность) • Единый набор архитектурных характеристик для всех модулей • Требуются усилия по поддержанию модульности (легко нарушить) • Эластичность и отказоустойчивость не на высоте

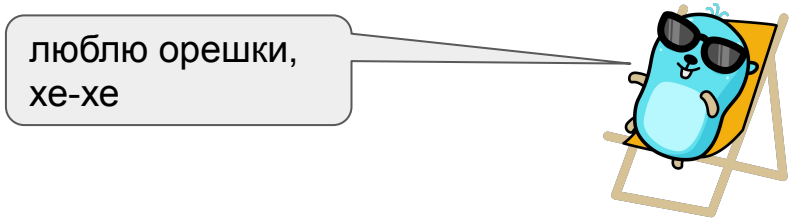
Модульный монолит - хороший старт!

- Строгое разделение областей, малая связанность / четкие API
- Монорепозиторий, простота разработки и отладки
- Простое развертывание



Микроядерная архитектура

- Позволяет легко создавать и добавлять новые возможности в виде плагинов
- Плагины могут быть разделены
  - по техническим признакам - слой бизнес-логики, слой хранения, ...
  - по предметным - плагин для алертов, плагин для отчетов
- Развертываются как правило как монолит, но могут быть и распределенными



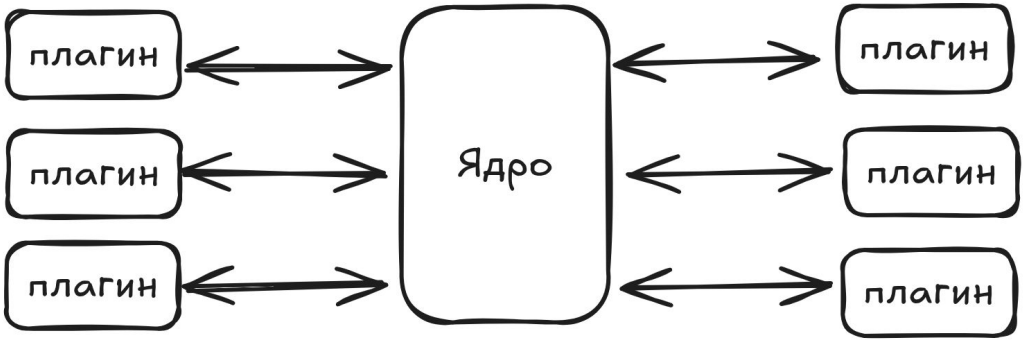
Микроядерная архитектура - состав

Ядро:

- минимум возможностей
- редко меняется после разработки
- не позволяет менять свое поведение (основной алгоритм)

Плагины:

- реализуют новые действия и конфигурации
- работают отдельно друг от друга
- развертываются независимо



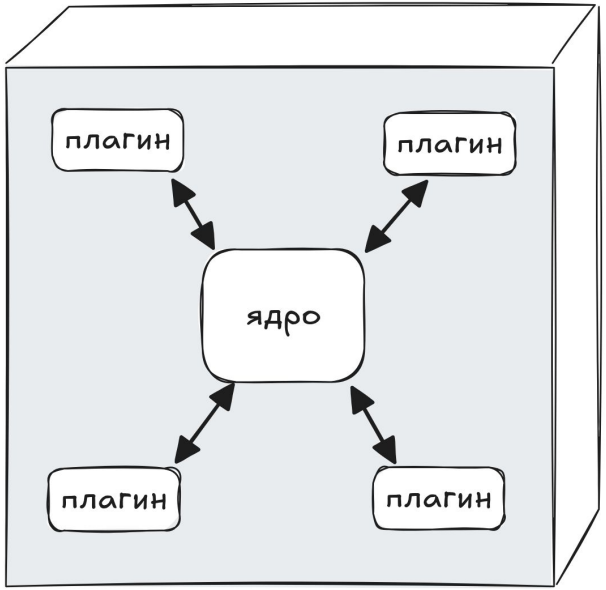
Микроядерная архитектура?

- Не все, что имеет плагины - микроядерная архитектура
- Степень микроядерности - неизменчивость и минимальные возможности.

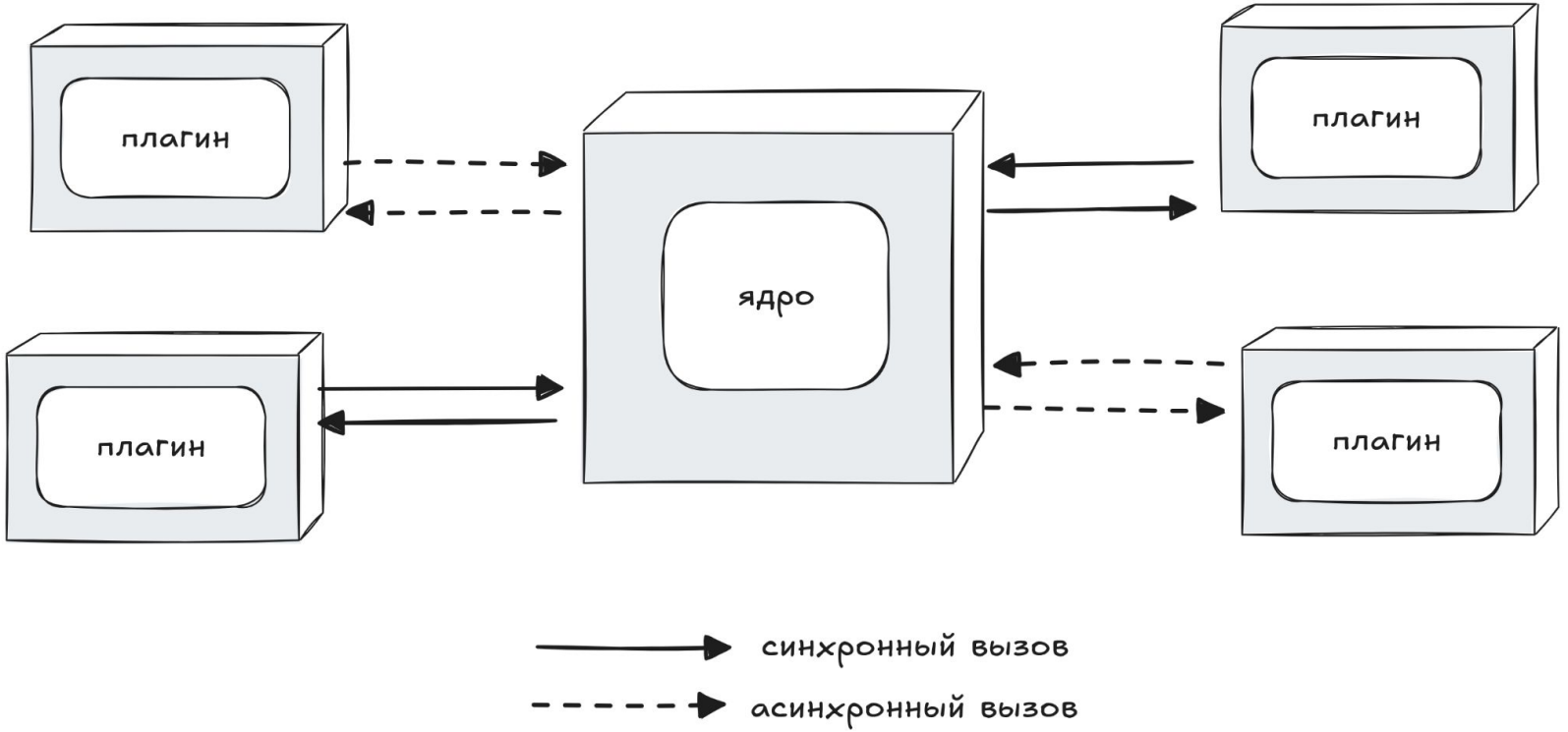


Монолитная микроядерная архитектура

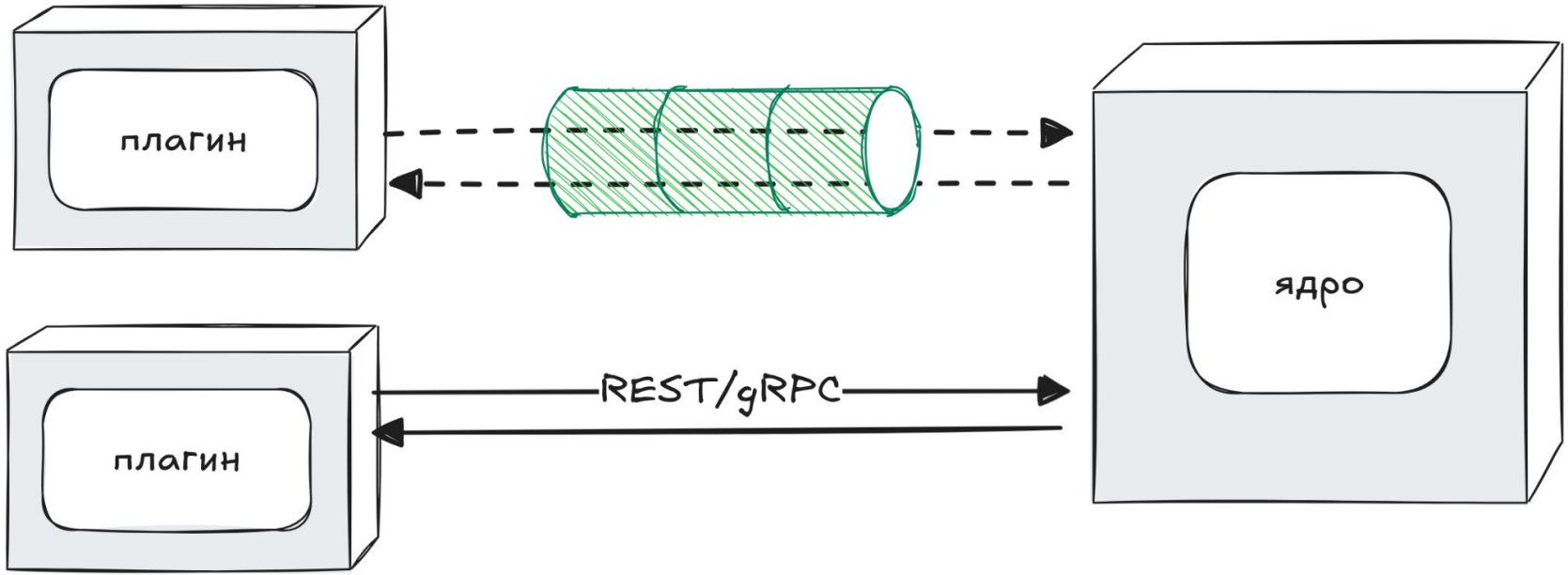
Плюсы	Минусы
● Производительность ● Простота	● Развертываемость ● Масштабирование



Распределенная микроядерная архитектура



Взаимодействие с ядром

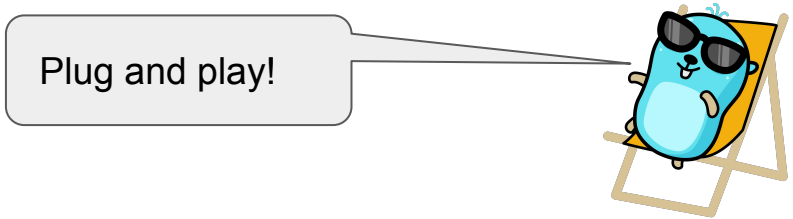


Распределенные плагины - компромиссы

Плюсы	Минусы
• Масштабируемость • Развертываемость (плагины отдельно от ядра) • Выбор языка программирования 😂 • Выбор стиля взаимодействия / компромиссов синхронной или асинхронной связи	• Производительность • Сложность

Компромиссы микроядерной архитектуры

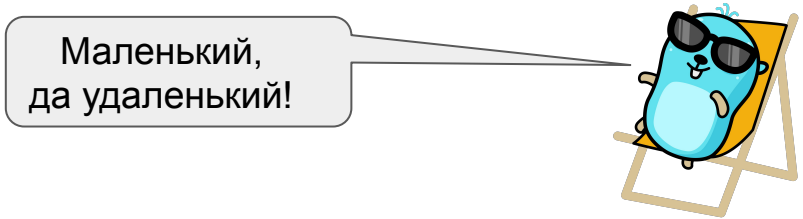
Сильные стороны	Слабые стороны
• Настройка поведения • Развиваемость • Разделение функций • Адаптируемость • Простая структура	• Общие зависимости плагинов • Неизменчивость ядра • Производительность (при физическом распределении) • Сложное взаимодействие плагинов (через ядро, версионирование контрактов)



Микросервисная архитектура

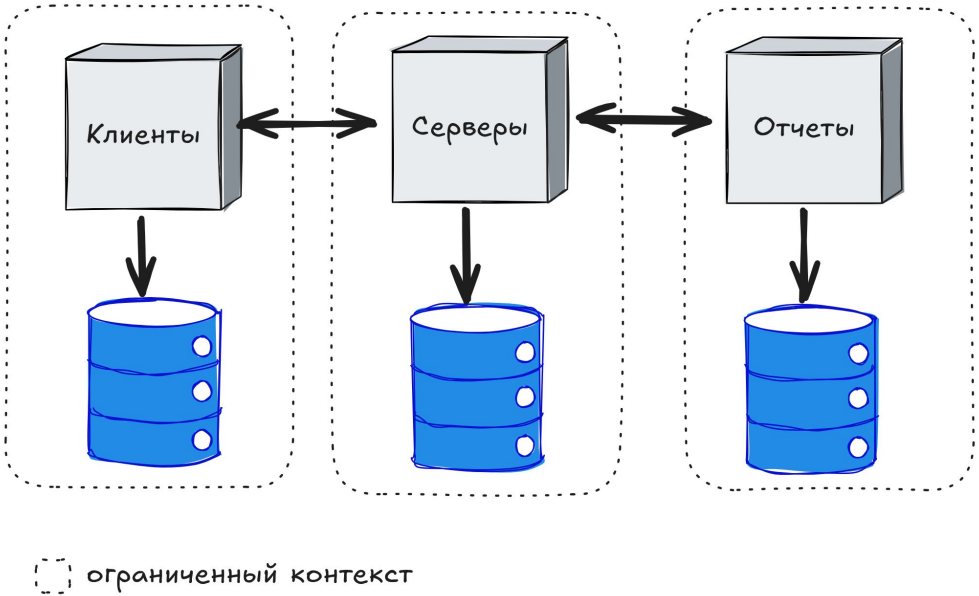
Сервис - отдельно развертываемая единица ПО, решающая бизнес-задачу (несколько функций).

Микросервис - отдельно развертываемая единица ПО, решающая одну задачу / функцию.



Ограниченный контекст

- Микросервис владеет своими данными (БД на микросервис)
- Основная причина - управление изменениями / их изоляция.



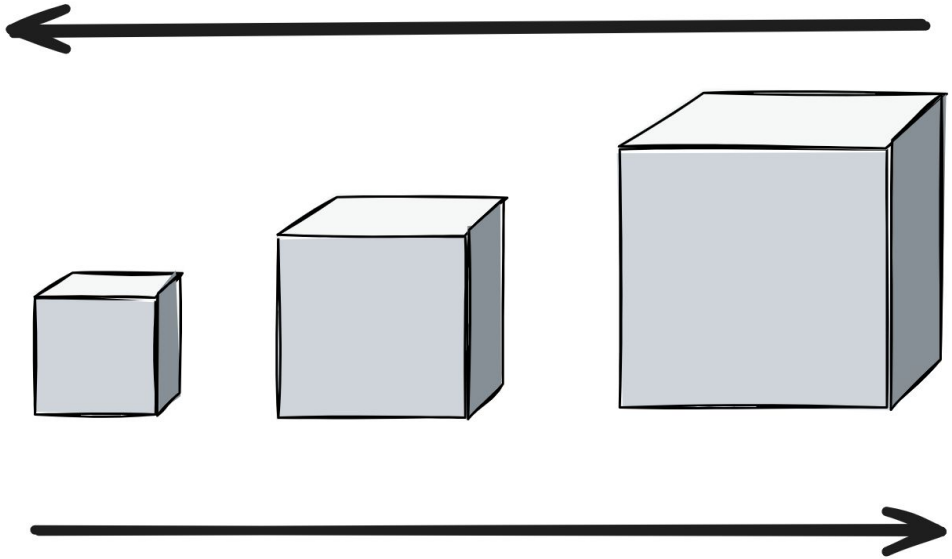
Декомпозиция микросервисов

Детализация - объем задач микросервиса.

- Избыточная детализация (нано-сервисы) - большая связанность, ухудшение производительности и надежности.
- Недостаточная детализация - крупные микросервисы сложно поддерживать, тестировать и масштабировать.
- Силы дезинтеграции дробят сервисы на части, силы интеграции - объединяют.

Силы дезинтеграции

Когда стоит уменьшить, разделить сервисы?



Силы интеграции

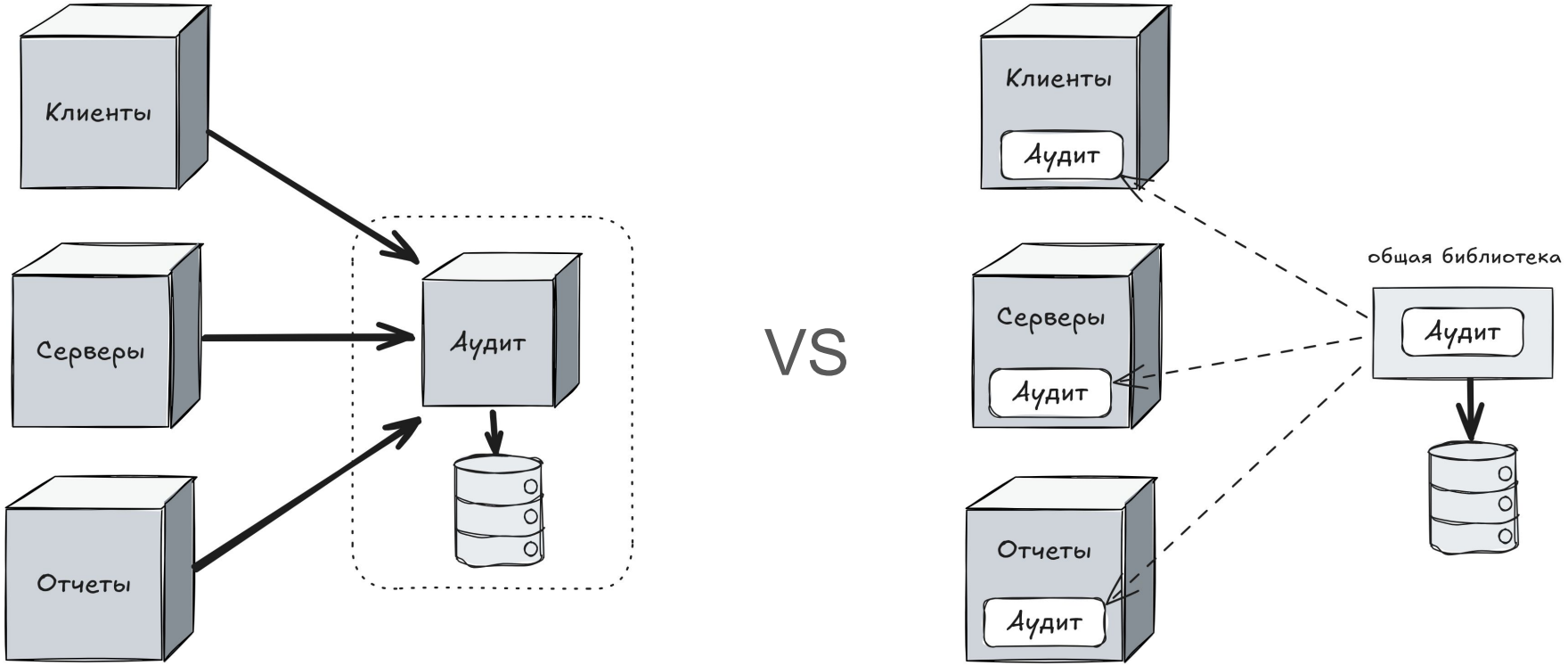
Когда стоит увеличить или объединить сервисы?

Силы интеграции и дезинтеграции

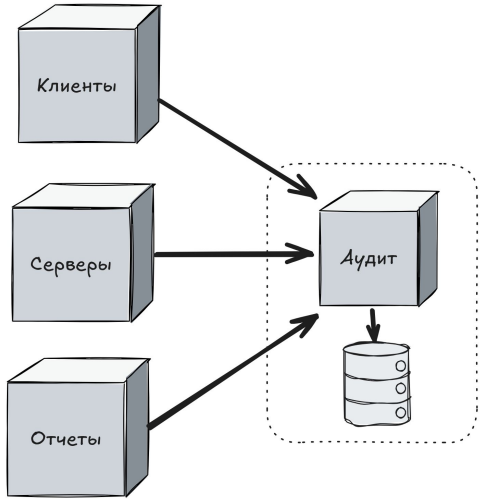
Необходимо разделение	Необходимо укрупнение
Несвязность функций	Транзакции в базе данных
Изменчивость кода	Общий код
Управление доступом	Зависимость данных
Отказоустойчивость и доступность	Рабочий процесс и хореография
Масштабируемость и пропускная способность	Производительность

Выбираем самый важный компромисс. Начинаем с "больших" микросервисов, делим если нужно.

Общая функциональность - сервис или библиотека?

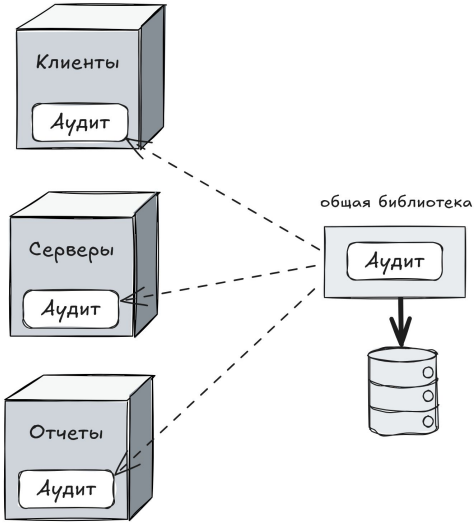


Общий сервис



Плюсы	Минусы
● Изменение кода не требует изменений в клиентах ● Сервис может быть написан на любом подходящем под задачу языке	● Риск изменения - влияет на всех клиентов ● Сетевые задержки ● Может пострадать доступность клиентов из-за недоступности сервиса ● Нужно масштабировать вместе с клиентами

Общая библиотека



Плюсы	Минусы
● Производительность ● Доступность ● Масштабируемость ● Меньше риск изменений за счет версионирования и обратной совместимости	● Несколько версий библиотек - под каждый язык программирования клиента ● Управление зависимостями, конфигами ● При изменении библиотеки надо тестировать все микросервисы-клиенты

Управление рабочими процессами

- Рабочий процесс
  - для обработки одного (бизнес-)запроса необходимо вызвать несколько микросервисов
- Способы управления рабочим процессом
  - оркестрация
  - хореография



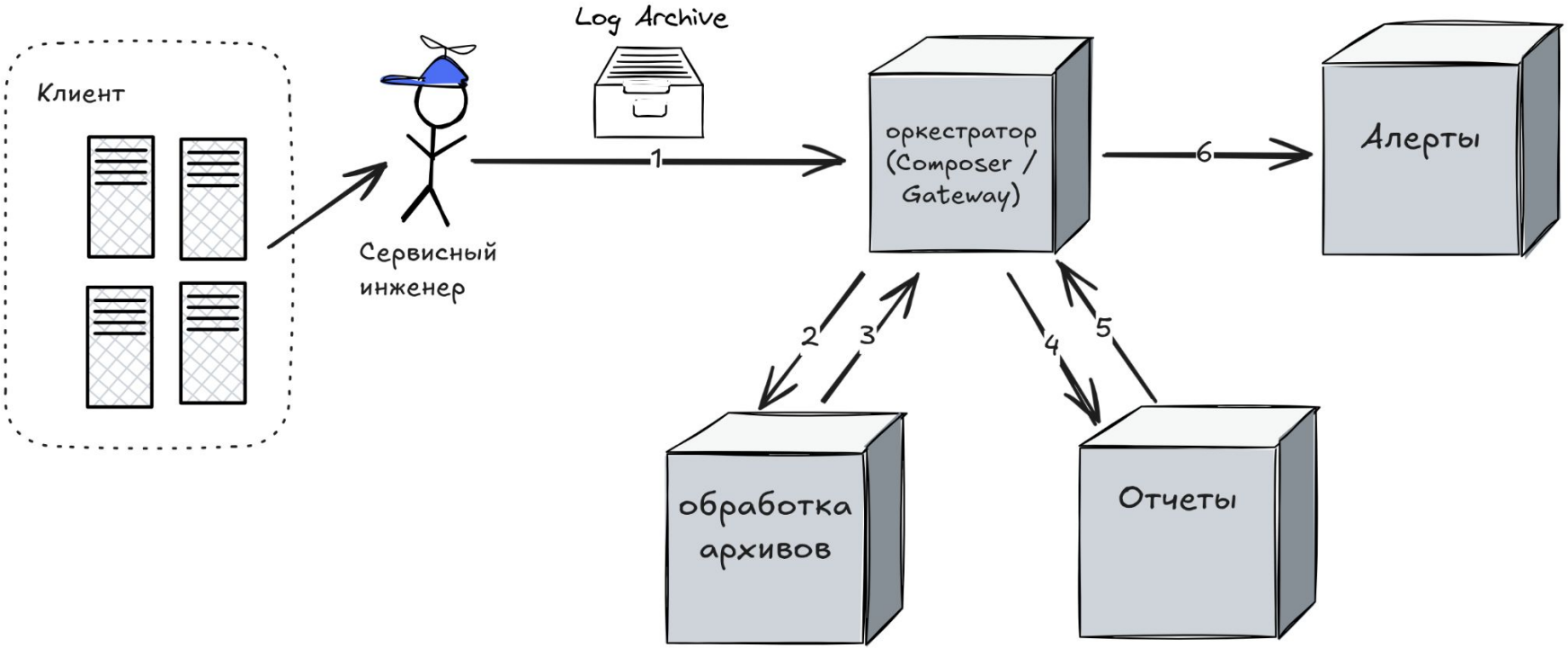


Оркестрация

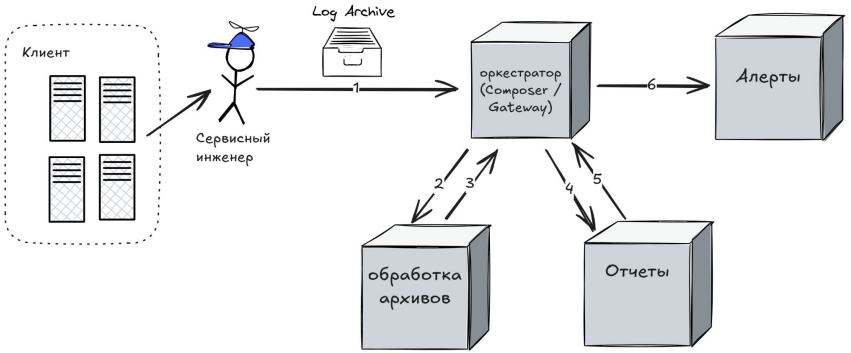
Координация действий микросервисов с помощью оркестратора, централизованного микросервиса:

- вызов микросервисов в правильном порядке
- отслеживание текущего состояния
- объединение данных из микросервисов
- обработка ошибок

Оркестрация



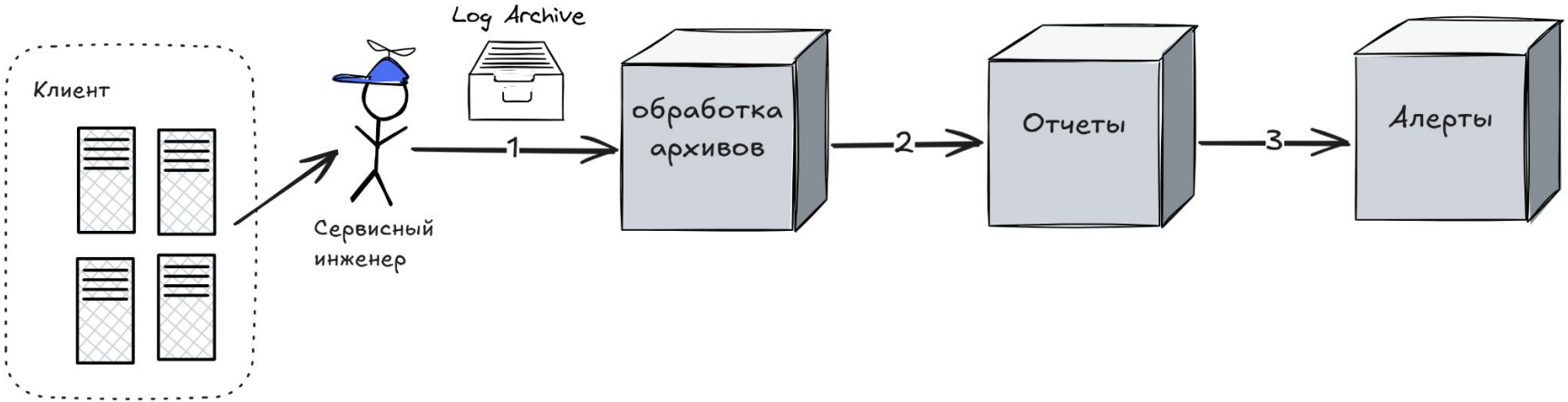
Оркестрация - за и против



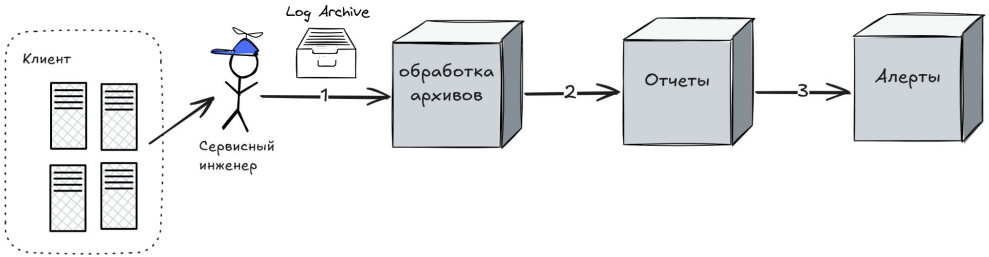
Плюсы	Минусы
● Централизация рабочего процесса ● Состояние процесса ● Обработка ошибок ● Изменения в рабочем процессе	● Производительность (гоняем данные, сохраняем состояние) ● Тесная связанность ● Масштабируемость ● Доступность

Хореография

Микросервисы по мере выполнения своей части рабочего процесса передают управление другим микросервисам.



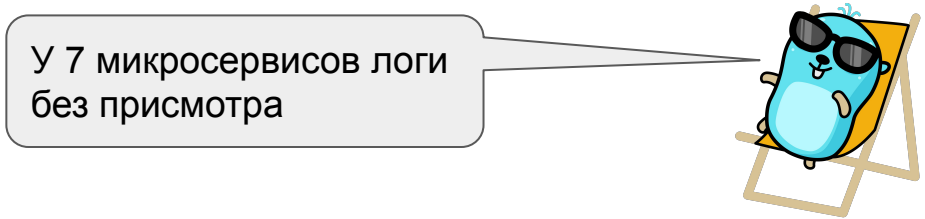
Хореография - за и против



Плюсы	Минусы
● Слабая связанность ● Отзывчивость ● Масштабируемость	● Обработка ошибок ● Восстанавливаемость ● Управление состоянием рабочего процесса

Компромиссы микросервисной архитектуры

Сильные стороны	Слабые стороны
• Удобство сопровождения • Тестируемость • Развертываемость • Развиваемость • Масштабируемость • Отказоустойчивость	• Сложность • Производительность • Не подходят для монолитных баз данных • Сложные рабочие процессы • Деление команд по техническим задачам



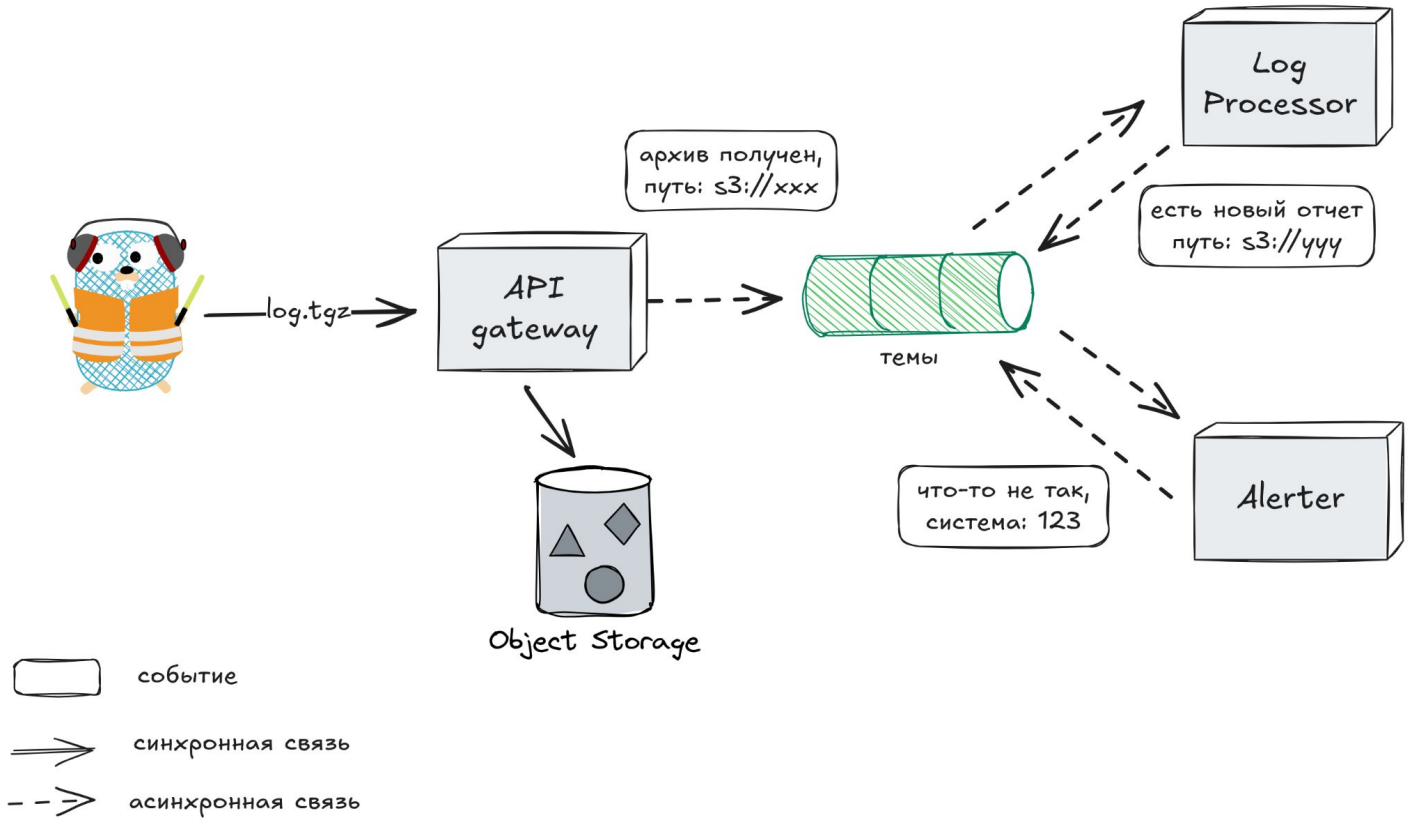
Событийно-управляемая архитектура

EDA - event driven architecture, события и их обработчики.

Главная идея - одновременная обработка данных по событиям (то что уже произошло) отдельными сервисами.

Сервисы общаются через шину событий асинхронно, не дожидаясь ответов.

Событийно-управляемая архитектура



Отличия сообщений от событий

Событие - то, что уже произошло:

- Сервис отправляет событие в канал и не ждет ответа
- Не знает, прочтет ли кто-то это событие (асинхронно, низкая связанность системы)
- Событие может содержать как полную информацию, так и идентификатор данных.

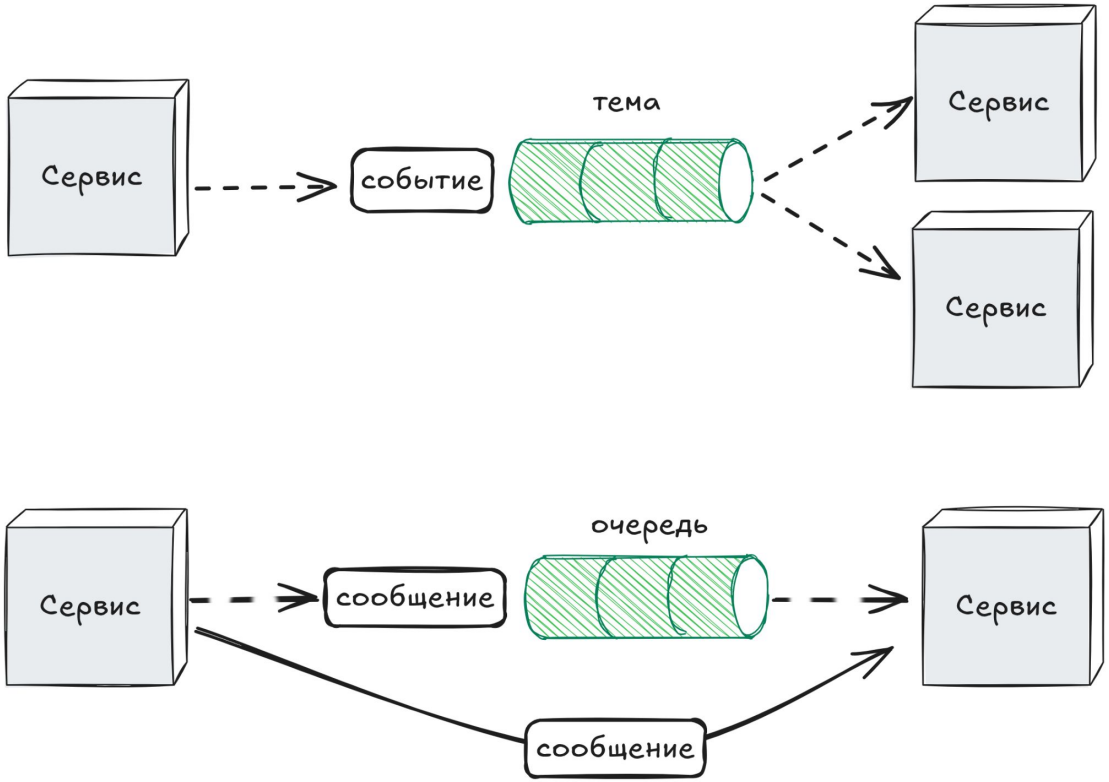
Сообщение - команда или запрос с ответом

- Происходит как правило синхронно
- Посылающий сервис знает адресата

Событие или сообщение?

Вид	Способ коммуникации	Что передается
Событие	асинхронная связь / канал тем	что-то уже произошло
Сообщение	синхронная связь, асинхронная связь / очередь сообщений	команды, запросы

Событие или сообщение?



Виды событий

События:

- Инициирующие - запросы от клиентов и внешних систем, таймеров и т.п.
- Производные - генерируются сервисами в ответ на инициирующие.

Любое действие порождает событие, даже если его никто не ждет, - расширяемость архитектуры.

Это дает возможность легко добавить новую функциональность (обработчик) позже.

Связанность в EDA

Слабая связанность в EDA на уровне сервисов - компромисс.

Для обмена данными сервисы

- используют общую базу
- синхронно просят данные друг у друга

В любом случае сильная связанность.



Способы организации баз данных в EDA

Характеристики	Монолитная БД	БД на предметную область	БД на сервис
Связанность сервисов	✅	❌	❌
Производительность	✅	✅	❌
Простота реализации	✅	✅	❌
Простота изменений	❌	?	✅
Отказоустойчивость	❌	❌	✅
Масштабируемость	❌	❌	✅

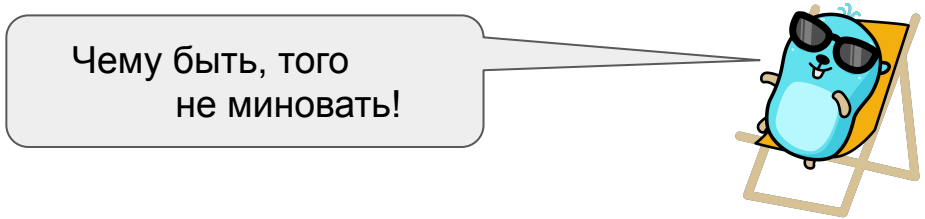
Отличия микросервисов и EDA

1. Производительность (EDA)
2. Ограниченные контексты (MSA)
3. Детализация данных
4. Детализация сервисов
5. Обработка событий vs сообщений
6. Стиль взаимодействия (синхронный / асинхронный)



Компромиссы событийно-управляемой архитектуры

Сильные стороны	Слабые стороны
• Удобство сопровождения • Производительность • Развиваемость • Масштабируемость • Отказоустойчивость	• Сложность • Тестируемость • Синхронные вызовы (портят отзывчивость) • Базы данных (связывают)



Журнал Архитектурных Решений (ADL)

ADR - Architecture Decision Record

- запись об архитектурном решении
- описывает какое решение было принято и почему
- помогает командам копить знания по принятых решениях (не только «как», но и «почему»)
- ADL (Log) - последовательно нумерованные решения, могут отменять и дополнять друг друга
- Принятые решения не изменяются, только отменяются следующими

Запись об Архитектурном Решении (ADR)

1. Заголовок
   - Номер-префикс и краткое название
2. Статус
   - Запрос на обсуждение (RFC)
   - Предложено
   - Принято
   - Отменено
3. Контекст
   - Задача, обстоятельства, факторы, возможные пути решения, анализ компромиссов
4. Решение
   - Что, как и почему
5. Последствия
   - Влияние на исполнителей, клиентов, инфраструктуру, риски, бюджет, время
   - Ограничения

Запись об Архитектурном Решении (ADR)

0007. API Gateway для клиентов поиска

Статус: предложено

Контекст: поисковая система используется REST клиентами, как сторонними API сервисами, так и "толстыми" клиентами. Необходимо обеспечить простое подключение, аутентификацию и авторизацию, rate limits, .. Рассмотренные решения и компромиссы ..

Решение: в системе поиска отдельный микросервис будет единой точкой входа для …

Последствия: single point of failure, повышенные требования к безопасности, ..

Рейтинги архитектурных стилей

	Многоуровневая архитектура	Модульный монолит	Микроядерная архитектура	Микросервисная архитектура	Событийно-управляемая архитектура
Удобство сопровождения	★	★★★	★★★	★★★★★	★★★
Тестируемость	★★	★★★	★★★	★★★★★	★★
Развертываемость	★	★★★	★★★	★★★★★	★★★
Простота реализации	★★★★★	★★★★	★★★★	★	★
Развиваемость	★	★★★	★★★	★★★★★	★★★★★
Производительность	★★ ★	★★★	★★★	★★	★★★★★
Масштабируемость	★	★	★★	★★★★★	★★★★★
Эластичность	★	★	★	★★★★	★★★★
Отказоустойчивость	★	★	★	★★★★★	★★★★★
Стоимость	$	$ $	$	$ $ $ $ $	$ $ $

чем больше ★, тем лучше

★ - в некоторых случаях (например, распределенные плагины)

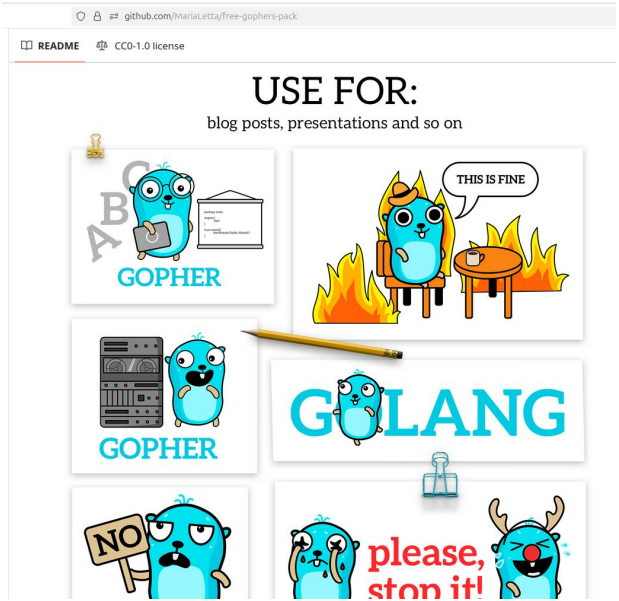
чем больше $, тем дороже

В слайдах задействованы

гоферы из стикер-пака Maria Letta:

https://github.com/MariaLetta/free-gophers-pack





Что почитать?





